Collaborative Gaussian SLAM for Multi-Robot Mapping: A Survey of Online Local-to-Global Systems

Prannaya Gupta[1]
[1]Pragnition Labs

Multi-robot Gaussian SLAM has emerged as a narrow but important variant of 3D Gaussian Splatting research: multiple independently moving robots or agents incrementally estimate local motion and local map state, then apply global alignment and global fusion to observations, local maps, Gaussian submaps, keyframes, or Gaussian primitives to form a global Gaussian or Gaussian-like map. This survey examines seven current systems: public or partly public implementation candidates MAGiC-SLAM, MAC-Ego3D, CoKo-SLAM, and CoMA-SLAM; and paper-only or not-yet-auditable systems HAMMER, GRAND-SLAM, and MAGS-SLAM. The main finding is architectural rather than leaderboard-based. Current systems are organized by fusion pattern—central-server global fusion, distributed Gaussian consensus, submap-plus-pose-graph consistency, monocular compact-submap fusion, and heterogeneous semantic global mapping—while benchmark evidence remains uneven. Accuracy and rendering metrics are commonly reported on Replica-derived multi-agent benchmarks, but communication, memory, merge latency, and scaling evidence are still sparse and not directly comparable across papers.

1. Introduction

3D Gaussian Splatting (3DGS) has quickly become a practical scene representation for differentiable rendering and dense visual mapping. In robotics, the hard question is not only whether a single camera can maintain a local Gaussian map, but whether multiple moving robots can contribute bounded online updates to a global map without collapsing under frame-alignment errors, duplicate primitives, network load, or implementation-specific runtime constraints.

This survey focuses on systems that satisfy a functional target: multiple independently moving agents, online or incremental operation, Gaussian or Gaussian-like local map state, a global fusion mechanism, and explicit cross-agent alignment. That target is narrower than ordinary 3DGS reconstruction and broader than papers whose title contains the word "SLAM." It includes RGB-D multi-agent Gaussian SLAM systems such as MAGiC-SLAM and GRAND-SLAM (Thomas et al., 2025a; Yugay et al., 2025), consensus-oriented systems such as MAC-Ego3D (Xu et al., 2024), communication-efficient compact-map systems such as CoKo-SLAM and CoMA-SLAM (Chen et al., 2026; Li et al., 2026), heterogeneous semantic mapping systems such as HAMMER (Yu et al., 2025a), and provisional monocular multi-agent systems such as MAGS-SLAM (Cao et al., 2026a).

2. Setting the Stage

Multi-robot SLAM is usually formulated as the joint estimation of robot trajectories and an environment representation from sensor measurements. Classical visual and visual-inertial systems express this through poses, landmarks, keyframes, and loop-closure factors; ORB-SLAM3 and Kimera-Multi are representative references for visual multi-map and distributed multi-robot evaluation practice (Campos et al., 2021; Tian et al., 2023). Gaussian SLAM changes the map variable. The global state is not only a sparse feature graph or occupancy map, but a differentiable set of Gaussian primitives

whose geometry, appearance, visibility, uncertainty, and provenance may all change during online collaboration.

Let the set of agents be

$$\mathcal{A} = \{1, \dots, N\}, \qquad N \geq 2,$$

and let $t \in \mathbb{N}$ index local update time. The paper uses $\mathcal{F}_0$ for a global or server frame and $\mathcal{F}_{i,t}$ for the current local camera, body, or active Gaussian-submap frame of agent i. A rigid transform

$$\mathrm{T}_{b \leftarrow a} \in SE(3)$$

maps a point expressed in frame $\mathcal{F}_a$ into frame $\mathcal{F}_b$. Thus $\mathrm{T}_{0 \leftarrow i,t}$ maps agent i's local coordinates at time t into the global frame. Local tracking estimates either this transform directly or a local-frame trajectory

$$\hat{\mathrm{X}}_{i,1:t} = \{\hat{\mathrm{T}}_{\ell_i \leftarrow i,1}, \dots, \hat{\mathrm{T}}_{\ell_i \leftarrow i,t}\},$$

where $\mathcal{F}_{\ell_i}$ is a local reference frame that must later be aligned to $\mathcal{F}_0$ by global tracking or global alignment.

The observation history for agent i is

$$\mathcal{Z}_i^{1:t} = \{z_{i,1}, \dots, z_{i,t}\}.$$

A single observation is method-dependent. For RGB-D systems one may write

$$z_{i,t}^{\mathrm{rgbd}} = (I_{i,t}, D_{i,t}, \mathrm{K}_i),$$

where $I_{i,t} : \Omega_i \to [0,1]^3$ is color, $D_{i,t} : \Omega_i \to \mathbb{R}_+$ is depth, and K_i contains intrinsics. Monocular systems omit $D_{i,t}$ and introduce an unknown or estimated scale $\lambda_{i,t} \in \mathbb{R}_+$. Heterogeneous systems may add local SLAM poses, LiDAR-derived depth, semantic embeddings, or device metadata,

$$z_{i,t}^{\mathrm{het}} = (I_{i,t}, \hat{\mathrm{T}}_{\ell_i \leftarrow i,t}, d_{i,t}, e_{i,t}, \xi_i),$$

where $d_{i,t}$ denotes optional geometric support, $e_{i,t}$ an optional semantic descriptor, and ξ_i a device or embodiment descriptor.

A Gaussian primitive is written as

$$g_k = (\boldsymbol{\mu}_k, \mathrm{R}_k, \mathrm{s}_k, o_k, \mathrm{c}_k, \mathrm{h}_k, \pi_k),$$

where $\boldsymbol{\mu}_k \in \mathbb{R}^3$ is the mean, $\mathrm{R}_k \in SO(3)$ and $\mathrm{s}_k \in \mathbb{R}_+^3$ parameterize anisotropic shape, $o_k \in [0,1]$ is opacity, c_k denotes color or spherical-harmonic appearance parameters, h_k is an optional semantic or learned descriptor, and π_k records provenance such as source agent, keyframe, or submap identity. A local map is a finite Gaussian or Gaussian-like set

$$\mathcal{M}_{i,t} = \{g_{i,t}^1, \dots, g_{i,t}^{m_{i,t}}\},$$

possibly partitioned into Gaussian submaps $\mathcal{B}_i^m \subseteq \mathcal{M}_{i,t}$, keyframe supports $\mathcal{K}_{i,t}$, or Gaussian-surfel submaps $\mathcal{S}_{i,t}$. A local tracking operator $\mathrm{local_tracking}_i$ estimates $\hat{\mathrm{X}}_{i,1:t}$ from observations and prior local state. A local mapping function is therefore

$$\mathrm{local_mapping}_i : \left(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t}\right) \mapsto \mathcal{M}_{i,t},$$

where $\mathcal{P}_{i,t}$ denotes method parameters such as insertion thresholds, photometric losses, depth weights, pruning rules, and keyframe policies.

The terminology in the remainder of the paper follows a local/global split. A *local map* is the full Gaussian state $\mathcal{M}_{i,t}$ maintained by agent i before cross-agent alignment. A *Gaussian submap* is a bounded partition $\mathcal{B}_i^m$ of a local map, while a *Gaussian-surfel submap* is denoted $\mathcal{S}_{i,t}$ when a method uses surfel primitives. The *global map* is always $\mathcal{G}_t$, whether it is held by a central server or distributed across agents. Local tracking and local fusion update $\hat{\mathrm{X}}_{i,1:t}$ and $\mathcal{M}_{i,t}$; global tracking, global alignment, and global fusion estimate cross-agent transforms and update $\mathcal{G}_t$.

Online collaboration requires bounded communication. Let

$$p_{i,t} = \mathrm{update_selection}_i(\mathcal{M}_{i,t}, \mathcal{K}_{i,t}, \hat{\mathrm{X}}_{i,1:t}; b_i)$$

be a packet selected under budget b_i. Depending on the method, $p_{i,t}$ may contain keyframes, descriptors, compacted Gaussians, Gaussian-submap summaries, loop-closure candidates, semantic embeddings, or local pose fragments. The active communication graph is $\mathcal{H}_t = (\mathcal{A}, \mathcal{R}_t)$; $\mathcal{N}_t(i) = \{j : (i,j) \in \mathcal{R}_t\}$ is the neighbor set for distributed methods.

Cross-agent alignment is represented by a constraint set

$$\mathcal{C}_t = \{c_{ab}\}, \qquad c_{ab} = (a, b, \Delta\mathrm{T}_{a\leftarrow b}, \mathbf{\Omega}_{ab}, \tau_{ab}),$$

where $\Delta\mathrm{T}_{a\leftarrow b} \in SE(3)$ is a relative transform, $\mathbf{\Omega}_{ab}$ is an information matrix or confidence surrogate, and τ_{ab} records the time, keyframe, or submap support. Constraints may arise from image retrieval, rendered-depth registration, camera-depth registration, geometric verification, semantic correspondence, consensus factors, or inter-robot loop closure. A generic alignment operator is

$$\mathrm{cross_agent_alignment}_t : \{p_{i,t_i}\}_{i\in\mathcal{A}} \mapsto \mathcal{C}_t.$$

The global map is the output of a global fusion operator

$$\mathcal{G}_t = \mathrm{global_fusion}_t\left(\{\mathrm{T}_{0\leftarrow i,t_i}\mathcal{M}_{i,t_i}\}_{i\in\mathcal{A}}, \mathcal{C}_t, \mathcal{P}_t\right),$$

where $\mathcal{P}_t$ contains global policies such as merge thresholds, update rates, consensus weights, redundancy tests, semantic-code retention, and communication schedules. Pose-graph methods instantiate $\mathrm{global_fusion}_t$ through an optimization over submap poses,

$$\{\mathrm{P}_v^\star\} = \arg\min_{\{\mathrm{P}_v\}} \sum_{e\in\mathcal{E}_t^{\mathrm{odom}}\cup\mathcal{C}_t} \rho\left(\|r_e(\{\mathrm{P}_v\})\|^2_{\mathbf{\Omega}_e}\right),$$

whereas consensus methods instantiate it through repeated local and inter-agent updates,

$$\mathcal{M}_{i,t}^{(h+1)} = C_{\mathrm{inter}}\left(\mathcal{M}_{i,t}^{(h)}, \{p_{j,t}^{(h)} : j \in \mathcal{N}_t(i)\}; \lambda_{\mathrm{inter}}\right).$$

Finally, reported performance should be read as a vector rather than a single score:

$$\mathrm{q} = \left(\mathrm{ATE}, \mathrm{PSNR}, \mathrm{SSIM}, \mathrm{LPIPS}, L_D, \ell_{\mathrm{track}}, \ell_{\mathrm{map}}, \ell_{\mathrm{merge}}, m_{\mathrm{GPU}}, s_{\mathrm{pkt}}, r_{\mathrm{net}}, N\right).$$

Here L_D is a depth error, ℓ_{track}, ℓ_{map}, and ℓ_{merge} are tracking, mapping, and merge latencies, m_{GPU} is peak memory, s_{pkt} is serialized update size, r_{net} is communication rate, and N is agent count. Current papers report different projections of q, which is why this survey emphasizes architecture rather than a universal ranking.

A system belongs to multi-robot online Gaussian SLAM when it satisfies five conditions: $N \geq 2$ independently moving agents; online or incremental operation; Gaussian or Gaussian-like local map

state; global-map semantics; and a cross-agent alignment mechanism. This excludes static multi-view 3DGS training, ordinary stereo, single-robot multi-camera rigs, and independent per-agent mapping without global Gaussian-map semantics.

The main algorithms below use named survey operators so that the comparison stays readable. Appendix A expands those operators into the tracking, mapping, communication, alignment, and fusion steps they summarize.

3. Existing Works

The following systems are summarized as survey-level procedures over the notation above. The algorithm boxes are not executable source code; they identify the main mathematical objects each method estimates, communicates, aligns, and fuses. System-specific tracking operators are used where a generic $\mathrm{local_tracking}_i$ call would hide materially different front ends.

3.1. MAGiC-SLAM

Mechanics. MAGiC-SLAM is a server-mediated RGB-D pipeline: agents perform local tracking, update local Gaussian maps, partition them into Gaussian submaps, send features or Gaussian submaps to a server, and the server performs global alignment plus global fusion before merging those submaps (Yugay et al., 2025).[1] The central mechanism is not raw image fusion; it is *Gaussian-submap plus loop-constraint* global fusion.

Algorithm 1 MAGiC-SLAM

Require: RGB-D streams $\{\mathcal{Z}_i^{1:t}\}_{i\in\mathcal{A}}$, local maps $\{\mathcal{M}_{i,t-1}\}$, Gaussian submaps $\{\mathcal{B}_i^m\}$, keyframe sets $\{\mathcal{K}_{i,t}\}$
Ensure: Global Gaussian map $\mathcal{G}_t$ and optimized Gaussian-submap poses $\{\mathrm{P}_v\}$

$\hat{\mathrm{T}}_{\ell_i\leftarrow i,t} \leftarrow \mathrm{magic_rgbd_tracking}_i(z_{i,t}, \mathcal{M}_{i,t-1}, \hat{\mathrm{X}}_{i,1:t-1})$ ▷ see Algorithm 13
$\hat{\mathrm{X}}_{i,1:t} \leftarrow \hat{\mathrm{X}}_{i,1:t-1} \cup \{\hat{\mathrm{T}}_{\ell_i\leftarrow i,t}\}$ ▷ trajectory extension
$\mathcal{M}_{i,t} \leftarrow \mathrm{local_mapping}_i(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t})$ ▷ local map update
$\mathcal{B}_i^m \subseteq \mathcal{M}_{i,t}$ ▷ submap policy refresh
$p_{i,t} \leftarrow \mathrm{update_selection}_i(\mathcal{M}_{i,t}, \mathcal{K}_{i,t}, \hat{\mathrm{X}}_{i,1:t}; b_i)$ ▷ server packet
$\mathcal{C}_t \leftarrow \mathcal{C}_{t-1} \cup \mathrm{cross_agent_alignment}_t(\{p_{i,t_i}\}_{i\in\mathcal{A}})$ ▷ verified loop constraints
$\{\mathrm{P}_v^\star\} \leftarrow \mathrm{pose_graph_optimize}(\mathcal{E}_t \cup \mathcal{C}_t)$ ▷ submap pose graph
$\mathcal{G}_t \leftarrow \mathrm{global_fusion}_t(\{\mathrm{P}_v^\star \mathcal{B}_v\}, \mathcal{C}_t, \mathcal{P}_t)$ ▷ global submap merge

Evidence and caveats. Reported ReplicaMultiagent results include about $0.27\,\mathrm{cm}$ ATE RMSE, $34.26\,\mathrm{dB}$ PSNR, $1.12\,\mathrm{GiB}$ peak GPU memory, $54\,\mathrm{MB}$ submaps, and $167\,\mathrm{s}$ global merge time. These figures make MAGiC-SLAM a mature public research baseline, but they also show that global merging can dominate runtime.

3.2. MAC-Ego3D

Mechanics. MAC-Ego3D replaces a single authoritative merge step with Gaussian consensus. Each RGB-D agent maintains a local Gaussian map; nearby Gaussians are regularized within an agent, and asynchronous inter-agent consensus aligns and refines the global structure (Xu et al., 2024).[2] The key object is the repeated consensus update, not a one-shot server merge.

[1] https://github.com/VladimirYugay/MAGiC-SLAM
[2] https://github.com/Xiaohao-Xu/MAC-Ego3D

Algorithm 2 MAC-Ego3D

Require: RGB-D streams $\{\mathcal{Z}_i^{1:t}\}$, local Gaussian maps $\{\mathcal{M}_{i,t-1}\}$, communication graph $\mathcal{H}_t$, consensus round h
Ensure: Consensus-refined local maps $\{\mathcal{M}_{i,t}^{(h+1)}\}$ and eventually consistent global map $\mathcal{G}_t$

$\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}, \hat{\mathrm{X}}_{i,1:t} \leftarrow \mathrm{mac_consensus_tracking}_i(\mathcal{Z}_i^{1:t}, \mathcal{M}_{i,t-1})$ ▷ see Algorithm 15
$\tilde{\mathcal{M}}_{i,t} \leftarrow \mathrm{local_mapping}_i(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t})$ ▷ provisional map
$\mathcal{M}_{i,t}^{\mathrm{loc}} \leftarrow C_{\mathrm{intra}}(\tilde{\mathcal{M}}_{i,t}; \lambda_{\mathrm{intra}})$ ▷ intra-agent consensus
$p_{i,t}^{(h)} \leftarrow \mathrm{update_selection}_i(\mathcal{M}_{i,t}^{\mathrm{loc}}, \hat{\mathrm{X}}_{i,1:t}, h; b_i)$ ▷ neighbor packet
$\mathcal{R}_{i,t}^{(h)} \leftarrow \{p_{j,t}^{(h)} : j \in \mathcal{N}_t(i)\}$ ▷ asynchronous exchange
$\mathcal{M}_{i,t}^{(h+1)} \leftarrow C_{\mathrm{inter}}(\mathcal{M}_{i,t}^{\mathrm{loc}}, \mathcal{R}_{i,t}^{(h)}; \lambda_{\mathrm{inter}})$ ▷ inter-agent consensus
$\mathcal{G}_t \leftarrow \{\mathcal{M}_{i,t}^{(h+1)} : i \in \mathcal{A}\}$ ▷ distributed global state

Evidence and caveats. Extracted results report about $0.14\,\mathrm{cm}$ ATE RMSE on Multi-Agent Replica, $40.04\,\mathrm{dB}$ PSNR, $0.02\,\mathrm{s}$ tracking, and $0.05\,\mathrm{s}$ mapping. Communication is reported as intervals rather than bytes.

3.3. CoKo-SLAM

Mechanics. CoKo-SLAM keeps the server pattern but changes what is transmitted: compact keyframe-optimized Gaussian/keyframe state instead of dense raw streams (Li et al., 2026).[3] The server uses compact map information, loop closure without an initial relative-pose guess, redundant-Gaussian pruning, and global optimization.

Algorithm 3 CoKo-SLAM

Require: RGB-D local maps $\{\mathcal{M}_{i,t}\}$, keyframes $\{\mathcal{K}_{i,t}\}$, budgets $\{b_i\}$, previous global map $\mathcal{G}_{t-1}$
Ensure: Compact globally optimized Gaussian map $\mathcal{G}_t$

$\hat{\mathrm{T}}_{\ell_i \leftarrow i,t} \leftarrow \mathrm{magic_rgbd_tracking}_i(z_{i,t}, \mathcal{M}_{i,t-1}, \hat{\mathrm{X}}_{i,1:t-1})$ ▷ inherited tracker; see Algorithm 13
$\hat{\mathrm{X}}_{i,1:t} \leftarrow \hat{\mathrm{X}}_{i,1:t-1} \cup \{\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}\}$ ▷ trajectory extension
$\mathcal{K}_{i,t} \leftarrow \mathrm{coko_keyframe_selection}(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{K}_{i,t-1})$ ▷ compact keyframe support
$\mathcal{M}_{i,t} \leftarrow \mathrm{local_mapping}_i(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t})$ ▷ local map update
$p_{i,t} \leftarrow \mathrm{update_selection}_i(\mathcal{M}_{i,t}, \mathcal{K}_{i,t}, \hat{\mathrm{X}}_{i,1:t}; b_i)$ ▷ compact packet; see Algorithm 19
$\mathcal{A}_t \leftarrow \{(a,b) : \mathrm{score}(p_{a,t}, p_{b,t}) \geq \tau_a\}$ ▷ supported associations
$\Delta\mathrm{T}_{a \leftarrow b} \leftarrow \mathrm{estimate_relative_alignment}(p_{a,t}, p_{b,t})$ for $(a,b) \in \mathcal{A}_t$ ▷ relative alignment
$\mathcal{D}_t \leftarrow \{g \in \cup_i \mathcal{M}_{i,t} : \mathrm{overlap}(g, \mathcal{G}_{t-1}) > \tau_o\}$ ▷ redundancy set
$\mathcal{U}_{i,t} \leftarrow \{g \in \mathcal{M}_{i,t} : g \notin \mathcal{D}_t\}$ ▷ surviving primitives
$\mathcal{G}_t \leftarrow \mathrm{global_fusion}_t(\{\mathrm{T}_{0 \leftarrow i,t}\mathcal{U}_{i,t}\}_{i \in \mathcal{A}}, C_t, \mathcal{P}_t)$ ▷ compact global fusion

Evidence and caveats. The paper reports 85–95% lower transmitted data, but the extracted evidence did not include absolute bandwidth. The public repository supports architectural inspection, but the available evidence is not sufficient for an independent reproducibility claim.

3.4. CoMA-SLAM

Mechanics. CoMA-SLAM is a distributed multi-agent Gaussian-surfel system: agents maintain Gaussian-surfel submaps, close intra/inter-agent loops, and run distributed keyframe optimization (Chen et al., 2026).[4] Its representation is 2D Gaussian surfels rather than conventional 3D radiance splats, so it

[3] https://github.com/lemonci/coko-slam
[4] https://github.com/npu-chenlin/CoMA-SLAM

belongs in a broad Gaussian-map survey with that caveat.

Algorithm 4 CoMA-SLAM

Require: Agent observations $\{\mathcal{Z}_i^{1:t}\}$, Gaussian-surfel submaps $\{\mathcal{S}_{i,t-1}\}$, neighbor sets $\{\mathcal{N}_t(i)\}$
Ensure: Distributed globally consistent Gaussian-surfel map $\mathcal{G}_t^{\text{surfel}}$

$\mathcal{S}_{i,t} \leftarrow \text{local_mapping}_i(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t})$ ▷ surfel submap update
$\mathcal{L}_t \leftarrow \text{candidates}(\mathcal{K}_{i,t}, \mathcal{S}_{i,t})$ ▷ intra/inter loops
$\mathcal{V}_t \leftarrow \{(k, \ell, \Delta\mathrm{T}_{k\leftarrow\ell}, \boldsymbol{\Omega}_{k\ell}, \tau_{k\ell})\}$ ▷ verified factors
$C_t \leftarrow C_{t-1} \cup \mathcal{V}_t$ ▷ constraint update
$p_{i,t} \leftarrow \text{update_selection}_i(\mathcal{S}_{i,t}, \mathcal{K}_{i,t}, C_t; b_i)$ ▷ neighbor packet
$p_{i,t} \leftrightarrow \{p_{j,t} : j \in \mathcal{N}_t(i)\}$ ▷ distributed exchange
$Q_t^\star \leftarrow \text{pose_graph_optimize}(C_t)$ ▷ keyframe/submap optimization
$\mathcal{G}_t^{\text{surfel}} \leftarrow \bigcup_i Q_t^\star \mathcal{S}_{i,t}$ ▷ aligned surfel state

Evidence and caveats. The AAAI evidence reports improved pose/rendering/geometric consistency and a 99.8% communication-bandwidth reduction claim, but exact numeric tables were not extracted. The repository was public, but no top-level license was found.

3.5. HAMMER

Mechanics. HAMMER is a server-side heterogeneous semantic mapper. Robots/devices stream local SLAM poses and visual/perception outputs; the server estimates global frame alignment without prior relative poses and trains or updates a metric-semantic global Gaussian map (Yu et al., 2025a,b). The distinctive output is not just geometry, but a semantic Gaussian map usable by downstream agents.

Algorithm 5 HAMMER

Require: Heterogeneous packets $u_{i,t} = (z_{i,t}, \hat{\mathrm{T}}_{\ell_i\leftarrow i,t}, e_{i,t}, \xi_i)$ and previous semantic map $\mathcal{G}_{t-1}$
Ensure: Metric-semantic global Gaussian map $\mathcal{G}_t$ and bounded query views $v_t(q)$

$u_{i,t} \leftarrow (z_{i,t}, \hat{\mathrm{T}}_{\ell_i\leftarrow i,t}, e_{i,t}, \xi_i)$ ▷ normalized packet
$\mathrm{A}_{0\leftarrow\ell_i} \leftarrow \text{estimate_device_alignment}(\{u_{i,\tau}\}_{\tau\le t}, \mathcal{G}_{t-1})$ ▷ device alignment
$\tilde{z}_{i,t} \leftarrow \mathrm{A}_{0\leftarrow\ell_i} \hat{\mathrm{T}}_{\ell_i\leftarrow i,t} z_{i,t}$ ▷ global-frame observation
$a_{i,t} \leftarrow \text{associate}(\tilde{z}_{i,t}, \mathcal{G}_{t-1})$ ▷ supported primitives
$(g_k, \mathrm{h}_k, \pi_k) \leftarrow \text{update}(a_{i,t}, \tilde{z}_{i,t}, e_{i,t})$ ▷ semantic Gaussian update
$\mathcal{G}_t \leftarrow \text{global_fusion}_t(\mathcal{G}_{t-1}, \{\tilde{z}_{i,t}\}, C_t, \mathcal{P}_t)$ ▷ server map update
$v_t(q) \leftarrow Q(\mathcal{G}_t, q)$ ▷ bounded query view

Evidence and caveats. Reported evidence includes Replica PSNR around $37.28\,\mathrm{dB}$, online updates around 15–$38\,\mathrm{ms/frame}$, alignment around $36\,\mathrm{s}$ per new robot, and 30–$35\,\mathrm{Mbps}$ per device in hardware tests. No public implementation was found.

3.6. GRAND-SLAM

Mechanics. GRAND-SLAM is best treated as Gaussian submaps plus a global consistency layer. Agents optimize RGB-D Gaussian submaps derived from their local maps; inter- and intra-robot loop closures create constraints; pose-graph optimization updates global-frame consistency (Thomas et al., 2025a,b). Do not describe it as CoKo-style centralized rendered-depth fusion unless a source explicitly supports that.

Algorithm 6 GRAND-SLAM

Require: RGB-D streams $\{\mathcal{Z}_i^{1:t}\}$, active Gaussian submaps $\{\mathcal{B}_i^m\}$, loop-candidate database
Ensure: Globally consistent Gaussian reconstruction $\mathcal{G}_t$

$\hat{\mathrm{T}}_{\mathcal{B}_i^m \leftarrow i,t} \leftarrow \mathrm{rgbd_submap_tracking}_i(z_{i,t}, \mathcal{B}_i^m)$ ▷ frame-to-submap pose; see Algorithm 14
$\mathcal{B}_i^{m+1} \leftarrow \mathrm{spawn}(\mathcal{B}_i^m)$ if drift, extent, or support exceeds threshold ▷ submap split
$\mathcal{B}_i^m \leftarrow \mathrm{optimize_gaussian_submap}(\mathcal{B}_i^m, \mathcal{K}_{i,t}^m)$ ▷ active submap optimization
$\mathcal{C}_t \leftarrow \mathrm{verify}(\text{intra/inter loop candidates})$ ▷ loop closures
$\{\mathrm{P}_v^\star\} \leftarrow \mathrm{pose_graph_optimize}(\mathcal{E}_t^{\mathrm{odom}} \cup \mathcal{C}_t)$ ▷ submap pose graph
$\mathcal{G}_t \leftarrow \bigcup_v \mathrm{P}_v^\star \mathcal{B}_v$ ▷ global reconstruction

Evidence and caveats. Extracted results include Multiagent Replica ATE around $0.25\,\mathrm{cm}$ and Kimera-Multi Outdoor six-agent evaluation over about $1.85\,\mathrm{km}$ with average ATE around $4.99\,\mathrm{m}$ (Tian et al., 2023). No public code was found in the research pass.

3.7. MAGS-SLAM

Mechanics. MAGS-SLAM is the RGB-only outlier: each monocular agent builds Gaussian submaps from its local map, transmits compact summaries, verifies cross-agent associations with geometry and appearance, and fuses via occupancy-aware Gaussian fusion (Cao et al., 2026a,b). The core problem is scale-aware collaboration without active depth.

Algorithm 7 MAGS-SLAM

Require: Monocular RGB streams $\{\mathcal{Z}_i^{1:t}\}$, local maps $\{\mathcal{M}_{i,t-1}\}$, Gaussian submaps $\{\mathcal{B}_i^m\}$, compact-summary budget b_i
Ensure: Scale-aligned collaborative global Gaussian map $\mathcal{G}_t$

$(\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}, \lambda_{i,t}) \leftarrow \mathrm{mags_monocular_tracking}_i(\mathcal{Z}_i^{1:t}, \mathcal{M}_{i,t-1})$ ▷ see Algorithm 16
$\hat{\mathrm{X}}_{i,1:t} \leftarrow \hat{\mathrm{X}}_{i,1:t-1} \cup \{\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}\}$ ▷ trajectory extension
$\mathcal{M}_{i,t} \leftarrow \mathrm{local_mapping}_i(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t})$ ▷ local map update
$\mathcal{B}_i^m \subseteq \mathcal{M}_{i,t}$ ▷ active submap refresh
$p_{i,t} \leftarrow \mathrm{update_selection}_i(\mathcal{B}_i^m, \mathcal{K}_{i,t}, \hat{\mathrm{X}}_{i,1:t}; b_i)$ ▷ compact summary
$\mathcal{A}_t \leftarrow \mathrm{verify}(p_{a,t}, p_{b,t}; \text{geometry, appearance, occupancy})$ ▷ accepted candidates
$(\lambda_i^\star, \Delta\mathrm{T}_{a \leftarrow b}^\star) \leftarrow \mathrm{scale_aware_alignment}(p_{a,t}, p_{b,t})$ ▷ scale-aware alignment
$\mathcal{G}_t \leftarrow \mathrm{occupancy_fusion}(\{\lambda_i^\star \mathrm{T}_{0 \leftarrow i,t} \mathcal{B}_i^m\}_i, \mathcal{O}_{t-1})$ ▷ occupancy-aware fusion

Evidence and caveats. MAGS-SLAM is the leading claimed monocular multi-agent 3DGS SLAM direction in this set, but available evidence is project-page/arXiv-stage. Code and data were reported as release-upon-acceptance, so runtime and bandwidth claims remain provisional.

4. Overall Architecture Taxonomy

The systems above can be compared by the factorization

$$\underbrace{\mathrm{local_tracking}_i}_{\text{local tracking}} + \underbrace{\mathrm{local_mapping}_i}_{\text{local mapping}} + \underbrace{\mathrm{update_selection}_i}_{\text{update selection}} + \underbrace{\mathrm{cross_agent_alignment}_t}_{\text{cross-agent alignment}} + \underbrace{\mathrm{global_fusion}_t}_{\text{global Gaussian fusio}}$$

The recurring question is not simply whether a method uses Gaussians. It is where the five responsibilities live, which variables cross the communication boundary, and which component absorbs drift, redundancy, and stale information.

4.1. Central-Server Submap and Gaussian Fusion

Central-server systems place most of $\mathrm{cross_agent_alignment}_t$ and $\mathrm{global_fusion}_t$ on a global mapper. MAGiC-SLAM transmits Gaussian submaps and loop-closure evidence to a server that optimizes Gaussian-submap poses before merging Gaussians. CoKo-SLAM keeps the same broad centralization but reduces $p_{i,t}$ through keyframe-optimized compact packets and redundant-Gaussian pruning. HAMMER also uses a server, but its packet stream is heterogeneous and semantic: the server aligns local SLAM outputs into a global frame and updates a metric-semantic global Gaussian map.

Algorithm 8 Central-server submap and Gaussian fusion pattern

Require: Local maps $\{\mathcal{M}_{i,t}\}$, trajectories $\{\hat{\mathrm{X}}_{i,1:t}\}$, packets $\{p_{i,t}\}$, previous global map $\mathcal{G}_{t-1}$
Ensure: Server-side global Gaussian map $\mathcal{G}_t$ and transforms $\{\mathrm{T}_{0\leftarrow i,t}\}$

$\mathcal{M}_{i,t} \leftarrow \mathrm{local_mapping}_i(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t})$ ▷ local update
$p_{i,t} \leftarrow \mathrm{update_selection}_i(\mathcal{M}_{i,t}, \mathcal{K}_{i,t}, \hat{\mathrm{X}}_{i,1:t}; b_i)$ ▷ bounded upload
$\mathcal{C}_t \leftarrow \mathrm{cross_agent_alignment}_t(\{p_{i,t_i}\}_{i\in\mathcal{A}})$ ▷ cross-agent constraints
$\{\mathrm{T}_{0\leftarrow i,t}\} \leftarrow \mathrm{optimize}(\mathcal{C}_t, \mathcal{E}_t^{\mathrm{odom}})$ ▷ global-frame transforms
$\mathcal{D}_t \leftarrow \{g \in \cup_i \mathcal{M}_{i,t_i} : \mathrm{overlap}(g, \mathcal{G}_{t-1}) > \tau_o\}$ ▷ redundancy filter
$\mathcal{U}_{i,t_i} \leftarrow \{g \in \mathcal{M}_{i,t_i} : g \notin \mathcal{D}_t\}$ ▷ surviving evidence
$\mathcal{G}_t \leftarrow \mathrm{global_fusion}_t(\{\mathrm{T}_{0\leftarrow i,t_i}\mathcal{U}_{i,t_i}\}_{i\in\mathcal{A}}, \mathcal{C}_t, \mathcal{P}_t)$ ▷ server fusion

The advantage of this pattern is a coherent global optimization point. Server-side fusion can compare packets across agents, remove duplicate primitives, and maintain a single $\mathcal{G}_t$. The cost is that merge latency, memory, bandwidth, and single-server availability become first-order variables. MAGiC-SLAM's reported global merge time illustrates this cost, while HAMMER's hardware results are unusually explicit about per-device Mbps streams. Available ablation evidence is mostly about loop closure, alignment, compaction, or baseline comparisons rather than a complete systems ablation over bandwidth and agent count.

4.2. Distributed Gaussian Consensus

Distributed Gaussian consensus keeps every $\mathcal{M}_{i,t}$ active and uses peer updates to move those local maps toward consistency. MAC-Ego3D expresses this pattern most directly: intra-agent consensus regularizes each $\mathcal{M}_{i,t}$, while inter-agent consensus uses packets from $\mathcal{N}_t(i)$ to refine cross-agent map agreement. CoMA-SLAM follows a related distributed idea with Gaussian-surfel submaps, loop factors, and distributed keyframe optimization.

Algorithm 9 Distributed Gaussian consensus pattern

Require: Local maps $\{\mathcal{M}_{i,t}^{(h)}\}$, graph $\mathcal{H}_t$, neighbor packets $\{p_{j,t}^{(h)} : j \in \mathcal{N}_t(i)\}$
Ensure: Consensus-refined local maps $\{\mathcal{M}_{i,t}^{(h+1)}\}$ and distributed global state $\mathcal{G}_t$

$\mathcal{M}_{i,t}^{(h)} \leftarrow \mathrm{local_mapping}_i(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t})$ ▷ local Gaussian state
$\bar{\mathcal{M}}_{i,t}^{(h)} \leftarrow \mathcal{C}_{\mathrm{intra}}(\mathcal{M}_{i,t}^{(h)}; \lambda_{\mathrm{intra}})$ ▷ local regularization
$p_{i,t}^{(h)} \leftarrow \mathrm{update_selection}_i(\bar{\mathcal{M}}_{i,t}^{(h)}, \mathcal{C}_t, h; b_i)$ ▷ scheduled packet
$\mathcal{R}_{i,t}^{(h)} \leftarrow \{p_{j,t}^{(h)} : j \in \mathcal{N}_t(i)\}$ ▷ neighbor exchange
$\mathcal{C}_{i,t} \leftarrow \mathrm{cross_agent_alignment}_t(p_{i,t}^{(h)}, \mathcal{R}_{i,t}^{(h)})$ ▷ local constraint view
$\mathcal{M}_{i,t}^{(h+1)} \leftarrow \mathcal{C}_{\mathrm{inter}}(\bar{\mathcal{M}}_{i,t}^{(h)}, \mathcal{R}_{i,t}^{(h)}; \lambda_{\mathrm{inter}})$ ▷ consensus update

This pattern reduces dependence on a single fusion server and can naturally support asynchronous

collaboration. Its hard problems are convergence, conflict resolution, duplicate primitive handling, stale packets, and communication scheduling. Reported evidence supports strong tracking/rendering for MAC-Ego3D and large relative communication reductions for CoMA-SLAM, but byte-level communication, stale-update, and failure-mode ablations remain sparse in the available evidence.

4.3. Submap-Plus-Pose-Graph Consistency

Submap-plus-pose-graph systems treat dense Gaussian maps as local products whose global consistency is controlled by a graph over Gaussian-submap poses. GRAND-SLAM is the clearest example: RGB-D Gaussian submaps are optimized independently, loop closures add factors to $\mathcal{C}_t$, and pose-graph optimization updates the transforms that place Gaussian submaps into the global frame. MAGiC-SLAM also uses this pattern inside a centralized architecture, but GRAND-SLAM makes the Gaussian-submap consistency layer the main comparison unit.

Algorithm 10 Submap-plus-pose-graph consistency pattern

Require: Gaussian submaps $\{\mathcal{B}_i^m\}$, odometry factors $\mathcal{E}_t^{\mathrm{odom}}$, loop candidates, previous poses $\{\mathrm{P}_v\}$
Ensure: Globally consistent Gaussian-submap placement and global map $\mathcal{G}_t$

$\hat{\mathrm{T}}_{\mathcal{B}\leftarrow i,t} \leftarrow \mathrm{rgbd_submap_tracking}_i(z_{i,t}, \mathcal{B}_i^m)$ ▷ observation-to-submap pose
$\mathcal{B}_i^{m+1} \leftarrow \mathrm{spawn}(\mathcal{B}_i^m)$ if drift or support exceeds threshold ▷ submap node creation
$\mathcal{B}_i^m \leftarrow \mathrm{optimize_gaussian_submap}(\mathcal{B}_i^m, \mathcal{K}_{i,t}^m)$ ▷ submap optimization
$\mathcal{C}_t \leftarrow \mathrm{verify}$(intra/inter loop closures) ▷ relative-pose constraints
$(\mathcal{V}_t^{\mathrm{pg}}, \mathcal{E}_t^{\mathrm{odom}} \cup \mathcal{C}_t) \leftarrow \mathrm{graph}(\{\mathcal{B}_i^m\})$ ▷ pose graph
$\{\mathrm{P}_v^\star\} \leftarrow \mathrm{pose_graph_optimize}(\mathcal{E}_t^{\mathrm{odom}} \cup \mathcal{C}_t)$ ▷ global consistency
$\mathcal{G}_t \leftarrow \bigcup_v \mathrm{P}_v^\star \mathcal{B}_v$ ▷ overlap-reconciled map

The strength of this pattern is that it reuses a mature SLAM abstraction: dense maps can remain local while global corrections act through $\mathrm{P}_v^\star$. The main risk is that a corrected pose graph does not by itself remove duplicate or inconsistent Gaussian primitives. A useful ablation surface is loop closure: GRAND-SLAM reports weaker results without loop closure, which supports the claim that cross-agent consistency factors, not only local photometric mapping, drive the large-scale result.

4.4. Compact Communication and Monocular Submap Fusion

Compact communication appears in both RGB-D and monocular systems, but it becomes structurally central for monocular collaboration. CoKo-SLAM uses compaction to reduce transmitted Gaussian/keyframe state while preserving enough information for server-side loop closure and pruning. MAGS-SLAM goes further: each agent has only RGB observations, so $\mathrm{update_selection}_i$ must produce compact Gaussian-submap summaries that support both cross-agent verification and scale-aware alignment.

Algorithm 11 Compact communication and monocular submap fusion pattern

Require: Local maps $\{\mathcal{M}_{i,t}\}$, keyframes $\{\mathcal{K}_{i,t}\}$, scale hypotheses $\{\lambda_{i,t}\}$, packet budgets $\{b_i\}$
Ensure: Bandwidth-limited global map $\mathcal{G}_t$ and verified cross-agent alignments

$(\hat{\mathrm{X}}_{i,1:t}, \lambda_{i,t}) \leftarrow \mathrm{mags_monocular_tracking}_i(\mathcal{Z}_i^{1:t}, \mathcal{M}_{i,t-1})$ ▷ local pose and scale
$\mathcal{M}_{i,t} \leftarrow \mathrm{local_mapping}_i(\mathcal{Z}_i^{1:t}, \hat{\mathrm{X}}_{i,1:t}, \mathcal{P}_{i,t})$ ▷ local map
$s(x) \leftarrow \mathrm{info}(x)$ for $x \in \mathcal{K}_{i,t} \cup \mathcal{M}_{i,t}$ ▷ packet scoring
$p_{i,t} \leftarrow \mathrm{update_selection}_i(\mathcal{M}_{i,t}, \mathcal{K}_{i,t}, \hat{\mathrm{X}}_{i,1:t}; b_i)$ ▷ compact evidence
$\mathcal{A}_t \leftarrow \mathrm{verify}(p_{a,t}, p_{b,t})$ ▷ geometry/appearance checks
$(\lambda_i^\star, \Delta\mathrm{T}_{a\leftarrow b}^\star) \leftarrow \mathrm{scale_aware_alignment}(p_{a,t}, p_{b,t})$ ▷ scale-aware transform
$\mathcal{G}_t \leftarrow \mathrm{global_fusion}_t(\{\lambda_i^\star \mathrm{T}_{0\leftarrow i,t} \mathcal{M}_{i,t}\}_{i\in\mathcal{A}}, \mathcal{C}_t, \mathcal{P}_t)$ ▷ accepted fusion

The tradeoff is between informativeness and bandwidth. Small packets help online operation, but if $p_{i,t}$ omits the wrong geometry, appearance, or occupancy evidence, $\mathrm{cross_agent_alignment}_t$ cannot reliably recover the transform. Monocular systems add scale ambiguity, so the alignment problem includes both rigid transform and scale hypotheses. CoKo-SLAM reports large relative transmission reductions; MAGS-SLAM is important architecturally but its code/data and quantitative communication evidence remain provisional in the sources used here.

4.5. Heterogeneous Semantic Global Gaussian Mapping

HAMMER expands the global map from geometry and radiance to metric-semantic scene memory. In the notation above, the Gaussian descriptor h_k and provenance π_k are not auxiliary bookkeeping; they are part of what makes $\mathcal{G}_t$ useful to heterogeneous robots and downstream queries. The alignment operator $\mathrm{cross_agent_alignment}_t$ must reconcile device-specific local SLAM outputs, sensor modalities, and asynchronous packet timing before semantic Gaussian updates are meaningful.

Algorithm 12 Heterogeneous semantic global Gaussian mapping pattern

Require: Heterogeneous streams $\{z_{i,t}^{\mathrm{het}}\}$, local poses $\{\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}\}$, descriptors $\{e_{i,t}\}$, previous semantic map $\mathcal{G}_{t-1}$
Ensure: Metric-semantic global map $\mathcal{G}_t$ with provenance-aware query views

$z_{i,t}^{\mathrm{het}} \leftarrow (I_{i,t}, \hat{\mathrm{T}}_{\ell_i \leftarrow i,t}, d_{i,t}, e_{i,t}, \xi_i)$ ▷ heterogeneous encoding
$\mathrm{A}_{0 \leftarrow \ell_i} \leftarrow \mathrm{estimate_device_alignment}(\{z_{i,\tau}^{\mathrm{het}}\}_{\tau \le t}, \mathcal{G}_{t-1})$ ▷ device alignment
$\tilde{z}_{i,t} \leftarrow \mathrm{A}_{0 \leftarrow \ell_i} z_{i,t}^{\mathrm{het}}$ ▷ global-frame packet
$a_{i,t} \leftarrow \mathrm{associate}(\tilde{z}_{i,t}, \mathcal{G}_{t-1})$ ▷ supported primitives
$(\boldsymbol{\mu}_k, \mathrm{R}_k, \mathrm{s}_k, o_k, \mathrm{c}_k, \mathrm{h}_k, \pi_k) \leftarrow \mathrm{update}(a_{i,t}, e_{i,t})$ ▷ metric-semantic update
$\mathcal{G}_t \leftarrow \mathrm{resolve}(\mathcal{G}_{t-1}, a_{i,t}; \text{uncertainty, support, semantics})$ ▷ conflict handling
$v_t(q) \leftarrow Q(\mathcal{G}_t, q)$ ▷ bounded view

This pattern is attractive because robot teams rarely have identical sensors or roles. A global map that contains semantics, source provenance, and queryable local evidence can support navigation, inspection, and human-robot interaction more directly than a purely photometric reconstruction. The open ablation surface is correspondingly broader: papers need to isolate the value of semantic descriptors, frame-alignment design, device heterogeneity, and communication choices rather than reporting only rendering quality or depth error.

5. Discussion and Conclusion

The main lesson from current multi-robot Gaussian SLAM is that architecture matters more than a single metric row. Local tracking, local mapping $\mathrm{local_mapping}_i$, update selection $\mathrm{update_selection}_i$, global alignment $\mathrm{cross_agent_alignment}_t$, and global fusion $\mathrm{global_fusion}_t$ are separable responsibilities with different failure modes. A paper can report strong PSNR or ATE while still leaving open whether it scales in bandwidth, memory, merge latency, stale updates, or heterogeneous deployments.

A useful next step for the field is standardized systems reporting. Alongside ATE, PSNR, SSIM, LPIPS, and depth error, papers should report serialized packet size, communication rate, per-update latency, global merge latency, peak memory, map size, agent count, trajectory length, and behavior under delayed or dropped messages. These quantities should be separated by component: tracking should not be conflated with mapping, mapping should not be conflated with global alignment, and global alignment should not be conflated with final rendering quality.

The architecture taxonomy also suggests a modular research path. Central-server methods should

be stress-tested for merge bottlenecks and server failure; distributed consensus methods should be evaluated under asynchronous and stale communication; submap-plus-pose-graph methods should report how dense Gaussian primitives are reconciled after graph corrections; compact-submap methods should expose the information lost by compression; semantic global-map methods should report provenance and uncertainty, not only photometric fidelity.

Multi-robot online Gaussian SLAM is therefore no longer speculative, but it remains immature as a systems discipline. MAGiC-SLAM and MAC-Ego3D are strong public baselines for centralized and consensus-oriented designs; CoKo-SLAM and CoMA-SLAM highlight communication-efficient directions with reproducibility caveats; HAMMER, GRAND-SLAM, and MAGS-SLAM expand the design space toward heterogeneous semantics, large-scale consistency, and monocular collaboration. Future progress will likely come from treating these patterns as composable architectural choices and evaluating each choice against the operational constraints of local-to-global robotic mapping.

References

C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós. ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multi-map SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021. doi: 10.1109/TRO.2021.3075644.

Z. Cao, Q. Shao, S. Zhai, J. Zhang, A. Nguyen, and B. Huang. MAGS-SLAM: Monocular multi-agent gaussian splatting SLAM for geometrically and photometrically consistent reconstruction, 2026a. URL https://arxiv.org/abs/2605.10760.

Z. Cao, Q. Shao, S. Zhai, J. Zhang, A. Nguyen, and B. Huang. MAGS-SLAM project page. Project website, 2026b. URL https://mags-slam.github.io/. Project page states code and data will be released upon acceptance; do not cite as public implementation evidence.

L. Chen, Y. Su, J. Wang, P. Han, Z. Xia, S. Bu, K. Li, B. Hu, S. Meng, and G. Wang. CoMA-SLAM: Collaborative multi-agent gaussian SLAM with geometric consistency. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 2922–2929, 2026. doi: 10.1609/aaai.v40i4.37283. URL https://ojs.aaai.org/index.php/AAAI/article/view/37283.

M. M. Q. Li, P.-Y. Lajoie, J. Liu, and G. Beltrame. Compact keyframe-optimized multi-agent gaussian splatting SLAM, 2026. URL https://arxiv.org/abs/2604.00804.

A. Thomas, A. Sonawalla, A. Rose, and J. P. How. GRAND-SLAM: Local optimization for globally consistent large-scale multi-agent gaussian SLAM. *IEEE Robotics and Automation Letters*, 10(12): 13129–13136, 2025a. doi: 10.1109/LRA.2025.3623026. URL https://doi.org/10.1109/LRA.2025.3623026.

A. Thomas, A. Sonawalla, A. Rose, and J. P. How. GRAND-SLAM: Local optimization for globally consistent large-scale multi-agent gaussian SLAM, 2025b. URL https://arxiv.org/abs/2506.18885.

Y. Tian, Y. Chang, L. Quang, A. Schang, C. Nieto-Granda, J. P. How, and L. Carlone. Resilient and distributed multi-robot visual SLAM: Datasets, experiments, and lessons learned, 2023. URL https://arxiv.org/abs/2304.04362.

X. Xu, F. Xue, S. Zhao, Y. Pan, S. Scherer, and X. Huang. MAC-Ego3D: Multi-agent gaussian consensus for real-time collaborative ego-motion and photorealistic 3D reconstruction, 2024. URL https://arxiv.org/abs/2412.09723.

J. Yu, T. Chen, and M. Schwager. HAMMER: Heterogeneous, multi-robot semantic gaussian splatting. *IEEE Robotics and Automation Letters*, 2025a. doi: 10.1109/LRA.2025.3575235. URL https://arxiv.org/abs/2501.14147. IEEE metadata should be checked for final volume, issue, and page numbers before camera-ready submission.

J. Yu, T. Chen, and M. Schwager. HAMMER project page. Project website, 2025b. URL http://hammer-project.github.io. Use for videos, deployment examples, and code-availability status; code was not public in the research pass.

V. Yugay, T. Gevers, and M. R. Oswald. MAGiC-SLAM: Multi-agent gaussian globally consistent SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6741–6750, 2025. URL https://openaccess.thecvf.com/content/CVPR2025/html/Yugay_MAGiC-SLAM_Multi-Agent_Gaussian_Globally_Consistent_SLAM_CVPR_2025_paper.html.

A. Expanded Survey Operators

The main text uses compact operators to keep the system comparisons readable. The procedures in this appendix expand those operators at survey resolution. They are not executable implementations; they identify the variables and evidence flow that the cited systems expose in papers, diagrams, or public repositories. Exact code paths may differ from these abstractions, especially for paper-only systems or repositories whose tracker and mapper split does not match the paper notation.

A.1. RGB-D Gaussian Tracking

Algorithm 13 RGB-D Gaussian tracking family

Require: RGB-D observation $z_{i,t} = (I_{i,t}, D_{i,t}, \mathrm{K}_i)$, active Gaussian map or submap $\mathcal{M}_{i,t-1}$, previous local trajectory $\hat{\mathrm{X}}_{i,1:t-1}$

Ensure: Updated local pose $\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}$

$P_t \leftarrow \mathrm{backproject}(I_{i,t}, D_{i,t}, \mathrm{K}_i)$ ▷ colored point cloud

$\mathrm{T}^0_{t-1,t} \leftarrow \mathrm{dense_register}(P_{t-1}, P_t)$ ▷ frame-to-frame initialization

Freeze Gaussian geometry, opacity, and appearance in $\mathcal{M}_{i,t-1}$

$(\hat{I}(\mathrm{T}), \hat{D}(\mathrm{T}), \alpha(\mathrm{T})) \leftarrow \mathrm{render}(\mathcal{M}_{i,t-1}, \mathrm{T}^0_{t-1,t} \circ \mathrm{T})$

$M_\alpha \leftarrow \mathrm{opacity_mask}(\alpha)$ and $M_{\mathrm{inlier}} \leftarrow \mathrm{residual_mask}(\hat{I}, \hat{D}, I_{i,t}, D_{i,t})$

$\mathrm{T}^\star_{t-1,t} \leftarrow \arg\min_{\mathrm{T} \in SE(3)} \sum M_{\mathrm{inlier}} M_\alpha \left(\lambda_c \|\hat{I}(\mathrm{T}) - I_{i,t}\|_1 + (1-\lambda_c)\|\hat{D}(\mathrm{T}) - D_{i,t}\|_1\right)$

return $\hat{\mathrm{T}}_{\ell_i \leftarrow i,t} \leftarrow \mathrm{compose}(\hat{\mathrm{X}}_{i,1:t-1}, \mathrm{T}^\star_{t-1,t})$

Algorithm 13 summarizes the MAGiC-SLAM tracking family and CoKo-SLAM's inherited tracker. The defining property is that pose refinement uses rendered color and rendered depth from a frozen Gaussian map, with masks for unreliable pixels, rather than updating Gaussians inside the tracking objective (Li et al., 2026; Yugay et al., 2025).

Algorithm 14 RGB-D submap tracking

Require: RGB-D observation $z_{i,t}$, active Gaussian submap $\mathcal{B}_i^m$
Ensure: Frame-to-submap pose $\hat{\mathrm{T}}_{\mathcal{B}_i^m \leftarrow i,t}$
1: Initialize pose from odometry, constant-velocity motion, or dense registration
2: Render color and depth from $\mathcal{B}_i^m$ under the candidate pose
3: Refine the pose against RGB-D residuals over the active keyframe or current frame support
4: **return** $\hat{\mathrm{T}}_{\mathcal{B}_i^m \leftarrow i,t}$

Algorithm 14 is the generic frame-to-submap version used by submap-plus-pose-graph systems. It should not be read as claiming that all such systems use MAGiC-SLAM's exact masking, initialization, or optimizer.

A.2. MAC-Ego3D Gaussian-Consensus Tracking

Algorithm 15 MAC-Ego3D Gaussian-consensus tracking

Require: RGB-D history $\mathcal{Z}_i^{1:t}$, local Gaussian map $\mathcal{M}_{i,t-1}$, previous local trajectory $\hat{\mathrm{X}}_{i,1:t-1}$
Ensure: Local pose $\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}$ and trajectory $\hat{\mathrm{X}}_{i,1:t}$
1: $\mathcal{M}_{\text{cand}} \leftarrow \text{select_visible_gaussians}(\mathcal{M}_{i,t-1}, \hat{\mathrm{T}}_{\ell_i \leftarrow i,t-1})$
2: $X_{i,t} \leftarrow \text{extract_rgbd_geometry}(z_{i,t})$
3: $\mathcal{A}_{i,t} \leftarrow \text{associate}(X_{i,t}, \mathcal{M}_{\text{cand}})$
4: $\hat{\mathrm{T}}_{\ell_i \leftarrow i,t} \leftarrow \text{align_observations_to_map}(X_{i,t}, \mathcal{A}_{i,t})$
5: $\hat{\mathrm{X}}_{i,1:t} \leftarrow \hat{\mathrm{X}}_{i,1:t-1} \cup \{\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}\}$
6: **return** $(\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}, \hat{\mathrm{X}}_{i,1:t})$

Algorithm 15 deliberately names MAC-Ego3D's local stage differently from Algorithm 13. The paper frames local estimation through intra-agent Gaussian consensus and Gaussian association, while the checked public tracker code exposes point-cloud alignment and pose-graph optimization paths; RGB/depth rendering losses are clearly evidenced in mapping rather than necessarily in the tracker routine (Xu et al., 2024).

A.3. MAGS-SLAM Monocular Tracking

Algorithm 16 MAGS-SLAM monocular local tracking

Require: Monocular RGB history $\mathcal{Z}_i^{1:t}$, active local map $\mathcal{M}_{i,t-1}$, learned depth prior, local window $\mathcal{W}_{i,t}$
Ensure: Local pose $\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}$ and scale state $\lambda_{i,t}$
1: Run a DROID-style front end over the sliding co-visibility window $\mathcal{W}_{i,t}$
2: Refine keyframe poses $\{\mathrm{T}\}$ and depth maps $\{d\}$ with dense bundle adjustment
3: Adjust depth and scale with joint depth-scale adjustment against the learned depth prior
4: Seed or update local Gaussians from inverse-depth keyframes
5: Freeze the current submap if the keyframe or motion threshold is reached
6: Assign a per-submap $\mathrm{Sim}(3)$ correction for later graph optimization
7: **return** $(\hat{\mathrm{T}}_{\ell_i \leftarrow i,t}, \lambda_{i,t})$

Algorithm 16 is paper-level only: the available MAGS-SLAM evidence describes a monocular DROID/JDSA front end and local Gaussian-map optimization, while public implementation code was not verified in this survey pass (Cao et al., 2026a,b).

A.4. Local Mapping and Update Selection

Algorithm 17 CoKo-SLAM keyframe selection

Require: Observations $\mathcal{Z}_i^{1:t}$, trajectory $\hat{\mathrm{X}}_{i,1:t}$, previous keyframes $\mathcal{K}_{i,t-1}$
Ensure: Updated keyframe set $\mathcal{K}_{i,t}$
1: Extract the current frame descriptor with the method's feature encoder
2: Compare it with descriptors attached to existing keyframes in the active submap
3: Add the current frame when its nearest keyframe distance exceeds the selection threshold
4: **return** $\mathcal{K}_{i,t}$

Algorithm 17 captures CoKo-SLAM's keyframe-selection contribution separately from its inherited MAGiC-SLAM tracking routine (Li et al., 2026).

Algorithm 18 Local Gaussian mapping

Require: Observations $\mathcal{Z}_i^{1:t}$, local trajectory $\hat{\mathrm{X}}_{i,1:t}$, parameters $\mathcal{P}_{i,t}$
Ensure: Local Gaussian state $\mathcal{M}_{i,t}$
1: Insert new primitives where rendered support is weak or where new keyframes add evidence
2: Optimize geometry, opacity, appearance, and optional descriptors against the method's rendering or geometric losses
3: Apply method-specific pruning, compaction, surfel maintenance, or semantic update rules
4: Partition the result into active submaps, compact keyframes, or Gaussian-surfel supports as required
5: **return** $\mathcal{M}_{i,t}$

Algorithm 19 Update selection under a communication budget

Require: Local map $\mathcal{M}_{i,t}$, keyframes $\mathcal{K}_{i,t}$, trajectory $\hat{\mathrm{X}}_{i,1:t}$, budget b_i
Ensure: Packet $p_{i,t}$
1: Score candidate Gaussians, submaps, keyframes, descriptors, loop candidates, or semantic records by expected alignment and fusion value
2: Remove redundant or low-support candidates according to the method's compaction policy
3: Select the highest-value subset whose serialized cost is within b_i
4: **return** $p_{i,t}$

A.5. Cross-Agent Alignment and Global Fusion

Algorithm 20 Cross-agent alignment

Require: Packets $\{p_{i,t_i}\}_{i\in\mathcal{A}}$
Ensure: Verified constraints $\mathcal{C}_t$
1: Retrieve candidate overlaps by image descriptors, Gaussian summaries, semantic descriptors, or neighbor exchange
2: Verify candidates with geometry, appearance, rendered depth, camera depth, occupancy, or consensus residuals
3: Estimate relative transforms in $SE(3)$ or $\mathrm{Sim}(3)$ as required by the modality
4: Attach confidence, information, timestamp, and support metadata to accepted constraints
5: **return** $\mathcal{C}_t$

Algorithm 21 Global Gaussian fusion

Require: Aligned local maps or submaps $\{\mathrm{T}_{0\leftarrow i,t_i}\mathcal{M}_{i,t_i}\}$, constraints $\mathcal{C}_t$, global policies $\mathcal{P}_t$
Ensure: Global map $\mathcal{G}_t$
1: Optimize global or submap poses when constraints define a pose graph
2: Merge, concatenate, or consensus-update Gaussian primitives according to the architecture
3: Suppress duplicates using overlap, occupancy, support, or semantic conflict tests
4: Optionally refine the fused map with rendering, geometry, or semantic losses
5: Preserve provenance so later queries can identify source agents, keyframes, and submaps
6: **return** $\mathcal{G}_t$